Kevin Küster

Zusammenfassung: Datenbanken

Datenbanken

DBS

Definitionen

- Datenbanksystem (DBS): System zur Beschreibung, Verwaltung, Speicherung & Wiedergewinnung großer Datenmengen, das von mehreren Anwendungen gleichzeitig und unabhängig benutzt werden kann
  - ↳ DBS besteht aus: DB, DBMS
- Datenbank (DB): Menge der gespeicherten Daten + Beschreibung der Struktur (Schema)
- Datenbankmanagementsystem (DBMS): Programme für den Umgang mit Daten und Schema (Oracle, MySQL, Postgre)
- Schema: langfristig stabil, definiert Struktur
- Daten: kurzfristig änderbar, genügen dem Schema
- Sprachen:
  - ↳ DDL (Data Definition Language) → Schema erstellen/ändern
  - ↳ DML (Data Manipulation Language) → Daten eintragen/ändern/löschen/abfragen

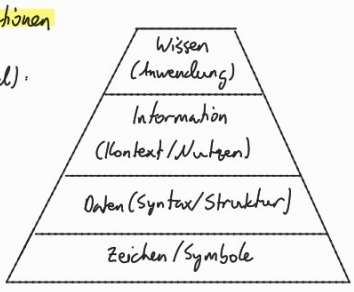
Datenbankmodelle

Historisch & praktisch relevant:

(i) Hierarchisches Modell (1960er): Baumstruktur z.B. IMS (IBM)
(ii) Netzwerkmodell (1960er): Gerichteter Graph
(iii) Relationales Modell (1970er, E.F. Codd): Tabellen/Relationen → Standard
(iv) Entity-Relationship-Modell: Konzeptuelles Entwurfsmodell (Hilfsmittel)
(v) Objekt-relationales Modell: Erweiterung des relationalen Modells um OOP-Konzepte
(vi) XML-Datenbankmodell: Semi-strukturierte Daten

Wissenspyramide & Abstraktionen

Wissenspyramide (aufsteigend):



3-Ebenen-Schema-Architektur (ANSI/SPARC):
- ↳ Externe Ebene: Benutzersicht (Views)
- ↳ Konzeptionelle Ebene: Logische Gesamtstruktur (Schema)
- ↳ Interne Ebene: Physische Speicherung

Datenunabhängigkeit (DU):
- ↳ Logische DU: Änderung konzeptuelles Schema → externe Sichten unberührt
- ↳ Physische DU: Änderung Speicherung → logisches Schema unberührt

Entity-Relationship

ER-Modellierung

Entity-Relationship-Modell (nach Chen, 1976): Konzeptuelles Modell für den Datenbankentwurf

Konzepte:
- Entity: Repräsentiert reales Objekt, von anderen unterscheidbar durch Attribute
- Attribut: Eigenschaft einer Entity, ordnet Wert aus Wertebereich zu
- Primärschlüssel: Minimale Attributmenge, die Entities eines Typs eindeutig identifizieren (im ER unterstrichen)
- Entity-Typ: Menge gleichartiger Entities (im ER Rechteck)
- Relationship (Beziehung): Zusammenhang zwischen zwei oder mehr Entities (im ER Raute)
- Beziehungstyp: Menge gleichartiger Beziehungen

Graphische Notation:
- Rechteck: Entity-Typ
- Raute: Beziehungstyp
- Ellipse: Attribut
- Unterstreichung: Schlüsselattribut
- Doppelter Rand: Schwacher (abhängiger) Entity-Typ

Kardinalitäten

Zweistellige Beziehungstypen klassifiziert nach Kardinalität:
- 1:1-Beziehung: Jede Entity aus E1 steht höchstens zu einer Entity aus E2 in Beziehung und umgekehrt
  Angestellter ←1:1→ Personalakte
- 1:n-Beziehung: Jede Entity aus E2 steht höchstens zu einer Entity aus E1, aber eine Entity aus E1 kann mit mehreren aus E2 in Beziehung stehen
  Firma ←1:n→ Angestellter
- m:n-Beziehung: Entities aus E1 können mit mehreren aus E2 in Beziehung stehen und umgekehrt
  Aktionär ←m:n→ Firma

Beteiligung:
- ↳ Obligatorisch (mandatory): Jede Entity muss an einer Beziehung teilnehmen (Punkt an Kante)
- ↳ Fakultativ (optional): Teilnahme nicht zwingend

Entity-Typen

Schwacher (abhängiger) Entity-Typ: Hat keinen eigenen vollständigen Primärschlüssel → wird erst durch Zugehörigkeit zu einem starken Entitätstypen eindeutig identifiziert

Notation: Doppelt umrandetes Rechteck + doppelt umrandete Raute

Bsp.: Angestellter ← hat_Kind → Kind (Kind nur identifizierbar durch Angestellter + Vorname)

Spezielle Beziehungstypen:

↳ Generalisierung: Zusammenfassung mehrerer Entity-Typen mit gemeinsamen Attributen zu einem allgemeinen Typ (buttom-up): Disjunkt

↳ Spezialisierung: Bildung von Teilmengen eines Entity-Typs (top-down): Können überlappen

Aggregierung: Beziehungstyp + beteiligte Entity-Typen → neuer Entity-Typ höherer Ordnung (wenn Beziehung zu Beziehung nötig ist)

Materialisierung: Abstraktum → Konkretisierung. 1:n mit individueller Attributvererbung (Bsp.: Kursangebot → konkreter Kurs)

Heuristiken – ER-Entwurf

Vorgehen: (i) Entity-Typen identifizieren
(ii) Attribute + Schlüssel festlegen
(iii) Generalisierung / Spezialisierung / Materialisierung prüfen
(iv) Beziehungstypen identifizieren + Kardinalitäten

Heuristiken:

▷ H1: Entity-Typ nur wenn neben Schlüssel noch weitere Attribute besitzt (=> sonst als Attribut deklarieren)

▷ H2: Mehrwertige Attribute → Als eigener Entity-Typ

▷ H3: Attribut mit 1:n-Beziehung zu anderem Entity-Typ → Eigener Entity-Typ

▷ H4: Attribut dem Entity-Typ zuordnen, dessen Primärschlüssel es „am direktesten beschreibt"

▷ H5: Bei 1:n-Hierarchie Attribut möglichst weit „oben" zuordnen

▷ H6: Keine Verschlüsselungen als Werte (Ausnahme: etablierter Code)

Redundanz vermeiden → Keine doppelte Darstellung von Zusammenhängen

SQL

SQL-DDL

▷ CREATE TABLE name(
  spalte1 DATENTYP [NOT NULL] [PRIMARY KEY],
  spalte2 DATENTYP [DEFAULT wert]
  ...
  CONSTRAINT pk PRIMARY KEY (spalte1)
  CONSTRAINT fk FOREIGN KEY (spalteX) REFERENCES andere_Tab (spalteY)
);

▷ Datentypen: INT, VARCHAR(n), DATE, DECIMAL(p,s), BOOLEAN, TEXT

▷ ALTER TABLE name ADD spalte DATENTYP;

▷ DROP TABLE name

SQL DML

▷ SELECT [DISTINCT] spalten FROM tabelle [alias]
  [JOIN tabelle2 ON bedingung]
  [WHERE bedingung]
  [GROUP BY spalten]
  [HAVING bedingung]
  [ORDER BY spalte [ASC/DESC]];

▷ INSERT INTO tabelle (spalten) VALUES (werte);

▷ UPDATE tabelle SET spalte = wert WHERE bedingung;

▷ DELETE FROM tabelle WHERE bedingung;

▷ JOINS:

↳ INNER JOIN: Nur Zeilen mit Match in beiden Tabellen

↳ LEFT JOIN: Alle Zeilen links + passende rechts (NULL, wenn kein Match)

↳ Impliziter JOIN: From t1, t2 WHERE t1.x = t2.y

▷ Aggregatfunktionen: COUNT(*), SUM(x), AVG(x), MIN(x), MAX(x)
  => Zusammen mit Group By verwenden

▷ HAVING: Filter auf Aggregat-Ergebnis (nach Group By)

Relationale Algebra → SQL

σ (Selektion)	WHERE
π (Projektion)	SELECT spalten
β (Umbenennung	AS
⋈ (Join)	JOIN / WHERE
∪ (Vereinigung)	UNION
∩ (Schnitt)	INTERSECT
− (Differenz)	EXCEPT

Beispiel

β_{zutaten ← name}
  (π_{name, alkoholgehalt}
  (σ_{land = 'De' ∨ land = 'En'} (zutat)
))

→ SELECT name AS zutat, alkoholgehalt FROM zutat
  WHERE land = 'De' OR land = 'En'

Normaltormen

Funktionale Abhängigkeiten

Funktionale Abhängigkeit (FD) $A \to B$ bedeutet:
Wenn zwei Tupel denselben Wert für A haben, dann haben sie auch denselben Wert für B
⇒ A bestimmt B funktional

Schlüssel: Minimale Attributmenge, die ALLE anderen Attribute funktional bestimmt

Superschlüssel: Jede Obermenge eines Schlüssels (nicht notwendig minimal)

Schlüsselkandidat: Minimaler Superschlüssel

Normaltormen (NF) 1, 2, 3

▷ 1. NF:
↳ Alle Attribute atomar (keine Mengen, keine zusammengesetzten Werte)
↳ Jede Zelle enthält genau einen Wert
↳ Maßnahme: Zusammengesetzte Attribute aufspalten, Wiederholungsgruppen in eigene Relation

▷ 2. NF:
↳ 1NF erfüllt
↳ Jedes Nichtschlüsselattribut ist voll funktional abhängig vom gesamten Primärschlüssel
⇒ Relevant nur bei zusammengesetzten Schlüssel
↳ Maßnahme: Attribute die nur von einem Teil des Schlüssels abhängen; ⇒ eigene Relation

▷ 3NF:
↳ 2NF erfüllt
↳ Kein Nichtschlüsselattribut ist transitiv abhängig von Primärschlüssel
⇒ Keine Abhängigkeit: Schlüssel → X → Y
↳ Maßnahme: Transitive Abhängigkeiten in eigene Tabelle auslagern

Zerlegung in 3.NF

(i) Alle funktionalen Abhängigkeiten (FDs) sammeln
(ii) Schlüssel bestimmen
(iii) Für jede FD $X \to Y$, die 3NF verletzt:
⇒ Neue Relation $R(X, Y)$ mit X als Schlüssel
(iv) Prüfen ob der Schlüssel der Originalrelation noch in einer Teilrelation enthalten ist (sonst hinzufügen)
(v) Redundante Relationen entfernen

Beispiel

$R(A, B, C, D, E, F, G)$: FDs: $A \to BCDE$, $D \to EF$, $F \to G$; PK: $\{A\}$
Verletzung: $A \to D \to EF$, $A \to D \to E \to \dots$, $D \to F \to G$
⇒ $R1(A, B, C, D)$; $R2(D, E, F)$, $R3(F, G)$

Integritätsregeln

▷ Strukturelle Integritätsregeln: Im Datenbankmodell eingebaut
↳ Automatisch vom DBMS geprüft
▷ Semantische Integritätsregeln: Anwendungsspezifisch
↳ explizit definiert; Check via SQL

Funktion: Sichern Konsitenz der Datenbank:
▷ Domänenintegrität: Werte müssen im erlaubten Wertebereich liegen
▷ Entitätsintegrität: Primärschlüssel darf nicht NULL sein und eindeutig
▷ Referenzielle Integrität: Fremdschlüsselwerte müssen auf existierende Primärschlüssel verweisen oder NULL sein
▷ Benutzerdefinierte Integrität: Check-Constrains, Trigger etc.

ORM (Object-Relational Mapping)

→ Technik zur Abbildung von OO-Objekten auf relationale Tabellen
Bei Vererbung drei Strategien:
▷ Single Table Inheritance: Eine Tabelle für alle Klassen der Hierarchie
↳ pro: einfach, keine JOINs
↳ contra: Viele NULL-Werte, große Tabelle
▷ Table per Class: Eine Tabelle pro konkreter Klasse (mit allen geerbten Attributen)
↳ pro: Kein JOIN nötig
↳ contra: Redundant gespeicherte Attribute
▷ Table per Type (joined): Eine Tabelle pro Klasse (nur eigene Attribute) ⇒ JOINs für Vererbung
↳ pro: Keine Redundanz
↳ contra: JOINs bei Abfragen nötig

⇒ Konzept:
Klasse → Tabelle
Attribut → Spalte
Objekt → Tupel
Referenz → Fremdschlüssel
Vererbung → spezielle Strategien